



Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

“Snakes & Lenders”

Case Study Documentation

Submitted by:

Group 10

Cabral, Shikina Y.

Caparas, John Christian B.

Exconde, Matan John B.

Rivera, Geuel John D.

April 14, 2026

Submitted to:

Prof. Ria Sagum



I. Introduction and its Background

Snakes & Ladders is inspired by the classic game of Snakes & Ladders, but instead of relying solely on dice rolls, players earn points that allow them to buy snakes to hinder opponents. The goal remains to reach the end first, but victory now depends on smart planning and control rather than chance alone. This twist introduces a strategic layer where players must manage resources, decide when to attack, and anticipate opponents' moves.

II. Game Proper

a. Objective of the Game

The objective of Snakes & Ladders is to be the first player to reach the final tile of the board. Players must strategically use points to place snakes, avoid penalties, and navigate the board efficiently.

b. Discuss the Rule of the Game

1. Board Setup

- The game is played on a 10×10 board (100 tiles).
- Each tile has a corresponding point value.
- The board is randomized/refreshed every game.
- There are 7 ladders and initially 4 snakes placed on the board.

2. Player Movement

- Players take turns rolling a die to move forward.
- Landing on ladders moves the player up; landing on snakes moves the player down.

3. Point System

- Players earn points based on the tile they land on.
- Points are used as currency to buy snakes.
- Points will be deducted to a player if she/he lands on a 'bomb' (trap tile).

4. Buying and Placing Snakes

- Players can buy snakes using accumulated points.
- The price of a snake depends on:
 - The length of the snake (tiles it covers)
 - Base value
- A player can own a maximum of 3 snakes.
- The cost increases with each additional snake purchased.
- Snakes cannot be placed on tiles currently occupied by another player.

5. Special Tiles

- Some tiles may contain bombs that deduct points.
- If a player's points drop below zero (bankrupt), they return to the starting tile.

6. Winning Condition

- The first player to reach the final tile (Tile 100) wins the game.

Game Visualization (Not final UI):



III. Implementation of AI Algorithm

a. AI Algorithm Implemented in the Game

To handle the mixture of random luck (dice rolls) and strategic money management in the game, the AI agents will be trained using two specific algorithms: Proximal Policy Optimization (PPO) for long-term learning, and Expectimax Search for immediate mathematical decision-making.

1. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is an advanced Reinforcement Learning algorithm where an agent learns to make optimal decisions through trial and error within a simulated environment. The defining characteristic of PPO is its "clipping" mechanism. When the AI discovers a new, successful tactic, PPO mathematically prevents the agent from updating its core logic too drastically all at once. It forces the AI to take small, incremental steps toward the new strategy, which ensures steady, stable learning without accidentally "forgetting" how to play the game.

Because this game involves buying traps and managing money, it is too complex to hardcode every single rule (e.g., telling the AI exactly when to save or spend one at a time). Instead, the system uses Reinforcement Learning,



where the AI learns by playing the game millions of times. It receives "rewards" for winning or collecting money, and "penalties" for going bankrupt.

We specifically chose Proximal Policy Optimization (PPO) over other learning algorithms because of its stability. In a game with high randomness (dice rolls), an AI can get confused and randomly forget good strategies. PPO prevents this by ensuring the AI only updates its strategy in small, safe increments. This allows the agent to slowly discover advanced economic strategies, like starving opponents of resources without breaking its own logic.

Pseudocode: PPO Training Loop (Multi-Agent Context)

Start with an AI that plays completely randomly.
For thousands of game simulations:

// Step 1: Play and Observe

For every turn the AI takes:

Look at the board (player positions, wallet amounts, active snakes).

Pick an action (Roll the dice OR Buy a snake).

See what happens (Did the AI get closer to winning? Did it lose money?).

Save the result of that action in memory.

// Step 2: Evaluate Success

Look back at the memory of the game.

Calculate which actions led to good outcomes (Rewards)
and which led to bad outcomes (Penalties).

// Step 3: Update Strategy

Carefully adjust the AI's internal decision-making logic:

Make it slightly MORE likely to repeat the actions that earned rewards.

Make it slightly LESS likely to repeat actions that caused penalties.

(Crucial PPO Step: Limit how drastically the logic can change at once
to keep the AI stable).

2. Expectimax Search

Expectimax Search is a decision-making algorithm used in game theory, specifically designed for environments that involve elements of chance or randomness. Unlike traditional board game algorithms (such as Minimax) which assume every move is perfectly predictable, Expectimax accounts for unpredictable events, like the rolling of dice. It works by building a "probability tree" that looks ahead into the future, branching not only based on player choices but also on "chance nodes" (the mathematical probability of different random outcomes occurring). It then calculates the "Expected Value"—the average mathematical outcome of a specific move.



While PPO is great for learning long-term strategies, we also need an algorithm that can do basic math to calculate if a trap is worth buying right now.

Classic board game AI algorithms (like Minimax used in Chess) do not work here because they assume there is no luck involved. Since our game relies on dice, the AI cannot perfectly predict the future. We chose Expectimax because it is designed specifically for games with chance. Instead of trying to find a guaranteed win, Expectimax calculates the "Expected Value." It looks ahead and asks: "If I spend 100 coins on a trap right now, what is the exact probability (based on a 1 in 6 dice roll) that my opponent will actually land on it?"

Pseudocode: Expectimax Action Evaluation

```
Function CalculateBestMove(current_board_state):
```

```
    // Check the AI's Options
```

```
    If it is the AI's turn to make a decision:
```

```
        Test every possible move (e.g., Buy Snake A, Buy Snake B, or just Roll).
```

```
        For each move, simulate what the board will look like next.
```

```
        Pick the move that results in the highest average score.
```

```
    // Check the Dice's Options (The Luck Factor)
```

```
    If a player is about to roll the dice:
```

```
        There are 6 possible outcomes (rolling a 1, 2, 3, 4, 5, or 6).
```

```
        Since each roll has a 1/6 chance of happening, simulate all 6 futures.
```

```
        Calculate the average outcome of all these futures combined.
```

```
        Return this "average future score" to the AI so it knows what to expect.
```



b. Software Architecture

